

04

Component'lar & Modern Template Syntax

© Başlangıç · Sıfır → Junior

🔗 Bu modülde ne öğreneceğiz?

- ✓ Bir component'in anatomisini: `@Component`, template, stil ve class
- ✓ Standalone component'leri ve `imports` ile bağımlılık yönetimini
- ✓ Dört çeşit data binding'i (interpolation, property, event, two-way)
- ✓ Modern control flow'u: `@if`, `@for` (track), `@switch`, `@empty`
- ✓ `@let` ile template içi yerel değişkenleri
- ✓ Signal tabanlı `input()`, `output()`, `model()` ile bileşen iletişimini

Ön koşul: Modül 1–3. Artık Angular'ın **kalbine** giriyoruz. Bu modülün sonunda kendi component'lerini yazıp aralarında veri taşıyabilecek ve gerçek bir **ToDo App** yapabileceksin.



Ders 4.1

Component Anatomisi

Component, Angular uygulamasının **yapı taşıdır**: ekranın yeniden kullanılabilir bir parçasını (görünüm + davranış) tek bir birimde toplar. Bir component üç şeyden oluşur: **mantık** (TypeScript class), **template** (HTML) ve **stil** (CSS/SCSS).

Bir Component'in İskeleti

TYPESCRIPT

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-selam',           // HTML'de <app-selam> olarak kullanılır
  templateUrl: './selam.html',   // ya da inline: template: `...`
  styleUrls: ['./selam.css'],    // ya da inline: styles: `...`
})
export class Selam {
  ad = signal('Efsane');          // durum (state)

  degistir(yeni: string) {       // davranış (metot)
    this.ad.set(yeni);
  }
}
```

@Component Yapılandırması

Alan	Ne işe yarar?
<code>selector</code>	Component'in HTML etiketi (örn. <code><app-selam /></code>).
<code>template</code> / <code>templateUrl</code>	Görünüm (inline yazılır veya ayrı <code>.html</code> dosyasından).
<code>styles</code> / <code>styleUrl</code>	Stil (inline veya ayrı dosya). Stiller o component'e kapsanır (scoped).
<code>imports</code>	Component'in template'inde kullandığı diğer component/directive/pipe'ler (standalone).

❗ Stiller kapsanır (scoped)

Bir component'in CSS'i **sadece o component'i** etkiler; başka yerlere sızmaz. Angular bunu otomatik yapar. Yani `.baslik` stilin yan component'in `.baslik`'ini bozmaz — büyük rahatlık.

Inline mı, Ayrı Dosya mı?

Küçük component'lerde `template: `...`` (inline) pratiktir; büyüdükçe ayrı `.html` / `.css` dosyaları daha okunaklıdır. CLI (`ng g c`) varsayılan olarak ayrı dosyalar üretir.

Component'i Kullanmak

Bir component'i başka bir component'in template'inde, selector'ıyla kullanırsın (ve o component'i `imports` 'a eklersin — Ders 4.2):

HTML

```
<app-selam />
<!-- veya kapalı: <app-selam></app-selam> -->
```

Lifecycle'a Kısa Bir Bakış

Component'lerin bir **yaşam döngüsü** vardır (oluşma, güncellenme, yok olma). En çok duyacağın `ngOnInit` — component oluştuğunda bir kez çalışır (genelde başlangıç verisi yüklemek için). Signal'ler çağında çoğu işi `computed` / `effect` ile yaparsın; lifecycle'ı ilerleyen modüllerde derinleştireceğiz. Şimdilik “böyle bir şey var” demen yeterli.

Mini Quiz

1. Bir component hangi üç parçadan oluşur?
2. `selector` ne işe yarar?
3. Bir component'in CSS'i başka component'leri etkiler mi?
4. Component oluştuğunda bir kez çalışan lifecycle kancası hangisidir?

Cevaplar

1. Mantık (TypeScript class), template (HTML), stil (CSS/SCSS).
2. Component'in HTML etiketini tanımlar (örn. `<app-selam />`).
3. Hayır; stiller o component'e kapsanır (scoped), dışarı sızmaz.
4. `ngOnInit`.

Anti-Patterns

- ✗ **Dev component'ler yazmak.** Bir component çok iş yapıyorsa böl; küçük, tek iş odaklı parçalar yönetilebilirdir.
- ✗ `selector` 'a ön ek (prefix) koymamak. `app-` gibi ön ek, HTML/3. parti etiketleriyle çakışmayı önler.
- ✗ **Mantığı template'e doldurmak.** Karmaşık hesapları class'ta (metot/ `computed`) yap; template sade kalsın.

✔ Bu Dersten Çıkarılacaklar

- ✓ Component = mantık (class) + template (HTML) + stil (CSS), `@Component` ile bağlanır.
- ✓ `selector` etiketi, `template/templateUrl` görünümü, `styleUrl` stili belirler.
- ✓ Stiller component'e kapsanır (scoped); yan etkilere yol açmaz.
- ✓ Component'i selector'ıyla (`<app-x />`) kullanırsın.
- ✓ Lifecycle vardır; en sık `ngOnInit` (oluşumda bir kez).



Modern Angular'ın en büyük sadeleştirmelerinden biri **standalone component**'lerdir. Modül 3'te "NgModule yok" demiştik; işte bunun anlamı bu.

Eski Dünya: NgModule

Angular yıllarca component'leri **NgModule** denen kutulara koymayı isterdi: her component bir modülde **declare** edilir, ihtiyaçlar modül seviyesinde **imports** edilirdi. Bu çok fazla "boilerplate" (tekrar eden kalıp kod) demekti ve yeni başlayanların kafasını karıştırırdı.

Yeni Dünya: Standalone

Standalone bir component **kendi başına yeter**: bir modüle ait olmaz, ihtiyaç duyduğu diğer component/directive/pipe'leri **doğrudan kendi imports** dizisinde belirtir.

TYPESCRIPT

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';
import { GorevItem } from './gorev-item/gorev-item';

@Component({
  selector: 'app-gorevler',
  imports: [RouterLink, GorevItem], // ihtiyaçlar burada
  template: `
    <a routerLink="/anasayfa">Ana sayfa</a>
    <app-gorev-item ... />
  `,
})
export class Gorevler {}
```

❗ `standalone: true` yazmana gerek yok

Angular v17'de standalone varsayılan oldu, v19'dan itibaren `standalone: true` yazman bile **gerekmez** — zaten varsayılan. (Tersine, eski NgModule'e bağlı bir component için `standalone: false` yazılır; v21'de buna neredeyse hiç ihtiyacın olmaz.)

imports Dizisine Ne Konur?

- Template'te kullandığın **diğer standalone component'ler** (örn. `GorevItem`).
- **Directive'ler** (örn. `RouterLink`, `RouterOutlet`).
- **Pipe'lar** (örn. `DatePipe`, `CurrencyPipe` — `@angular/common` 'dan).
- Eski modüller de gerekirse (örn. form senaryosunda `FormsModule`).

🔔 Kural

Template’te bir component/directive/pipe kullanıyorsan ve Angular “bilinmiyor” diye hata veriyorsa, çoğu zaman onu `imports`’a eklemeyi unutmuşsundur.

Standalone’ın Faydaları

- **Az boilerplate:** Modül dosyaları, declarations, exports derdi yok.
- **Net bağımlılık:** Bir component’in neye ihtiyacı olduğunu tek bakışta görürsün.
- **Kolay lazy load:** Component’leri tek tek tembel yüklemek çok kolay (Modül 16).

🔔 Mini Quiz

1. Standalone component ne demektir?
2. Bir standalone component, template’inde kullandığı başka bir component’i nereye ekler?
3. v21’de `standalone: true` yazmak gerekir mi?
4. Standalone’ın iki faydasını söyle.

🔑 Cevaplar

1. Bir NgModule’e ait olmayan, ihtiyaçlarını kendi `imports`’unda belirten component.
2. Kendi `imports` dizisine.
3. Hayır; v19+’dan beri varsayılandır, yazmaya gerek yok.
4. Az boilerplate, net bağımlılık (ve kolay lazy load).

⊗ Anti-Patterns

- ⊗ **Yeni projede NgModule kurmak.** Standalone dururken gereksiz; eski örnekleri kopyalama.
- ⊗ `imports`’a eklemeyi unutmak. “Known component değil” hatalarının bir numaralı sebebi.
- ⊗ **Her şeyi tek dev component’te toplamak.** Standalone’ın amacı küçük, bağımsız parçalar; bunu boşa çıkarma.

☑ Bu Dersten Çıkarılacaklar

- ✓ Standalone component NgModule'e ihtiyaç duymaz; bağımlılıklarını kendi `imports` 'unda belirtir.
- ✓ v17'den beri varsayılan; v19+'da `standalone: true` yazmaya bile gerek yok.
- ✓ `imports` 'a: diğer component'ler, directive'ler, pipe'lar (ve gerekirse eski modüller).
- ✓ Faydaları: az boilerplate, net bağımlılık, kolay lazy load.
- ✓ "Bilinmeyen etiket" hatası genelde eksik `imports` demektir.



Ders 4.3

Data Binding — 4 Çeşit

Data binding, **class (mantık)** ile **template (görünüm)** arasındaki köprüdür. Dört temel çeşidi var; yönlerine göre düşün: veri class'tan ekrana mı gidiyor, ekrandan class'a mı, yoksa çift yönlü mü?

Çeşit	Sözdizimi	Yön	Ne için
1. Interpolation	<code>{{ ifade }}</code>	Class → Ekran	Metin gösterme
2. Property binding	<code>[özellik]="ifade"</code>	Class → Ekran	Element özelliği ayarlama
3. Event binding	<code>(olay)="metot()"</code>	Ekran → Class	Kullanıcı olayını yakalama
4. Two-way binding	<code>[(model)]="deger"</code>	Çift yön	Form girdileri

1) Interpolation — `<code class="inline">{{ }}</code>`

Bir ifadenin sonucunu metin olarak basar. Signal'leri parantezle çağırırsın:

```
HTML
<h1>Merhaba {{ ad() }}</h1>
<p>Toplam: {{ fiyat() * adet() }}</p>
```

2) Property Binding — `<code class="inline">[ ]</code>`

Bir HTML element/component **özellikliğini** bir ifadeye bağlar. Köşeli parantez “bu bir metin değil, bir ifade” demektir:

```
HTML
<button [disabled]="yukleniyor()">Kaydet</button>
<img [src]="kullanici().avatar" [alt]="kullanici().ad" />
<app-kart [baslik]="urun().ad" />
```

Class ve style için kısayollar da vardır:

```
HTML
<li [class.bitti]="gorev().tamamlandi">...</li>
<span [style.color]="aktif() ? 'green' : 'gray'">durum</span>
```


❓ Attribute vs property

Çoğu zaman `[prop]` yeterlidir. Nadiren gerçek bir HTML *attribute*'u ayarlamak gerekirse (örn. ARIA) `[attr.aria-label]="..."` kullanılır.

3) Event Binding — `<code class="inline">( )</code>`

Kullanıcı olaylarını (tık, girdi, tuş...) dinler ve bir metod çağırır. `$event` ile olay nesnesine erişirsin:

HTML

```
<button (click)="artir()">+</button>
<input (input)="ara($event)" />
<input (keyup.enter)="gonder()" />  <!-- tuş filtresi -->
```

4) Two-Way Binding — `<code class="inline">[( )]</code>`

Hem class'tan ekrana hem ekrandan class'a; ikisini birden. “Muz sepette” (banana in a box) `[( )]` olarak hatırlanır. Klasik form senaryosunda `FormsModule`'ün `ngModel`'i ile:

HTML

```
<!-- imports: [FormsModule] gerekir -->
<input [(ngModel)]="ad" />
<p>Yazdığın: {{ ad }}</p>
```

Kendi component'inde de `model()` ile two-way sağlayabilirsin (Ders 4.6): `<app-sayac [(deger)]="x" />`.

❓ Mini Quiz

1. Dört binding çeşidini ve yönlerini söyle.
2. `[disabled]="yukleniyor()"` hangi binding çeşidi?
3. Olay nesnesine template'te nasıl erişirsin?
4. Two-way binding sözdizimini nasıl hatırlanır?

🔑 Cevaplar

1. Interpolation (class → ekran), property (class → ekran), event (ekran → class), two-way (çift yön).
2. Property binding.
3. `$event` ile (örn. `(input)="ara($event)"`).
4. “Muz sepette”: `[(model)]` — köşeli parantez içinde parantez.

⊗ Anti-Patterns

- ✗ **Property binding'de parantezi unutmak.** `disabled="yukleniyor()"` metin sayılır; doğrusu `[disabled]="yukleniyor()"`.
- ✗ **Signal'i parantezsiz okumak.** `{{ ad }}` değil `{{ ad() }}`; signal fonksiyon gibi çağrılır.
- ✗ **ngModel için FormsModule'u import etmemek.** Aksi halde two-way çalışmaz, hata alırsın.

☑ Bu Dersten Çıkarılacaklar

- ✓ Interpolation `{{ }}` : ifadeyi metin olarak basar.
- ✓ Property binding `[prop]` : element/component özelliğini ifadeye bağlar (`[class.x]` , `[style.y]` kısayolları).
- ✓ Event binding `(olay)` : kullanıcı olayını metoda bağlar; olay verisi `$event` .
- ✓ Two-way `[(model)]` : çift yön; `ngModel` (FormsModule) veya kendi `model()` 'in ile.



Template’te koşul ve döngü kurmak için Angular’ın v17’de getirdiği **yerleşik control flow** sözdizimini kullanınız: `@if`, `@for`, `@switch`. Bunlar eski `*ngIf` / `*ngFor`’un yerini alır — daha okunaklı, daha hızlı ve **import gerektirmez** (dile gömülüdür).

@if / @else if / @else

HTML

```
@if (kullanici()) {  
  <p>Hoş geldin, {{ kullanici()!.ad }}</p>  
} @else if (yukleniyor()) {  
  <p>Yükleniyor...</p>  
} @else {  
  <p>Lütfen giriş yap.</p>  
}
```

@for — Listeler (track ZORUNLU)

Bir diziyi dönerek liste oluşturur. Her öğeyi benzersiz tanımlayan bir `track` ifadesi **zorunludur** (Angular değişiklikleri verimli uygulayabilsin diye):

HTML

```
<ul>  
  @for (gorev of gorevler(); track gorev.id) {  
    <li>{{ gorev.baslik }}</li>  
  } @empty {  
    <li>Henüz görev yok.</li>  
  }  
</ul>
```

İçeride hazır **implicit değişkenler** bulunur: `$index`, `$first`, `$last`, `$even`, `$odd`, `$count`. Bunlara takma ad verebilirsin:

HTML

```
@for (urun of urunler(); track urun.id; let i = $index) {  
  <p>{{ i + 1 }}. {{ urun.ad }}</p>  
}
```

⚠️ `<track>` neden zorunlu?

`track`, Angular'a "hangi öğenin hangisi olduğunu" söyler. Doğru track ile liste değişince Angular tüm DOM'u yeniden çizmez, sadece değişeni günceller — büyük listelerde ciddi performans. Benzersiz bir kimlik (genelde `id`) kullan.

@switch / @case / @default

Tek bir değere göre birden çok dal arasında seçim yapar (çok sayıda `@if`'ten temiz):

```
@switch (durum()) {
  @case ('yukleniyor') { <p>Yükleniyor...</p> }
  @case ('hazir')      { <app-liste /> }
  @case ('hata')       { <p>Bir hata olustu.</p> }
  @default             { <p>Bilinmeyen durum.</p> }
}
```

HTML

Neden Eski `*ngIf`/`*ngFor` Değil?

Eski yapısal directive'ler (`*ngIf`, `*ngFor`, `*ngSwitch`) artık **deprecated** (v20). Yeni control flow daha okunaklı, daha performanslı ve `@angular/common`'dan import gerektirmez. Eski örneklerde `*ngIf` görürsen: sadece tanı, ama yeni kodda `@if` kullan. (Angular'ın migration aracı eskiyi otomatik çevirebilir.)

🔍 Mini Quiz

- `@for`'da hangi ifade zorunludur ve neden?
- Liste boşken bir şey göstermek için hangi blok kullanılır?
- `@for` içindeki hazır değişkenlerden üçünü say.
- Eski `*ngIf` / `*ngFor`'un durumu nedir?

🔑 Cevaplar

- `track` zorunludur; Angular değişiklikleri verimli uygulayıp gereksiz yeniden çizimi önler.
- `@empty` bloğu.
- `$index`, `$first`, `$last` (ayrıca `$even` / `$odd` / `$count`).
- Deprecated (v20); yeni kodda `@if` / `@for` kullanılır.

⊗ Anti-Patterns

- ✗ `@for` 'da `track` unutmak/yanlış kullanmak. `track $index` yerine benzersiz `id` kullan; yoksa performans ve durum hataları olur.
- ✗ Yeni kodda `*ngIf` / `*ngFor` yazmak. Deprecated; `@if` / `@for` kullan.
- ✗ Çok sayıda iç içe `@if` ile bir değeri kontrol etmek. Tek değere bağlıysa `@switch` daha temiz.

☑ Bu Dersten Çıkarılacaklar

- ✓ `@if` / `@else if` / `@else` : koşullu gösterim, import gerektirmez.
- ✓ `@for (x of liste()); track x.id) : track` zorunlu; `@empty` ile boş durum.
- ✓ Hazır değişkenler: `$index` , `$first` , `$last` , `$even` , `$odd` , `$count` .
- ✓ `@switch` / `@case` / `@default` : tek değere göre çok dallı seçim.
- ✓ Eski `*ngIf` / `*ngFor` deprecated; yeni control flow tercih edilir.



Ders 4.5

@let ile Template Değişkenleri

Bazen aynı ifadeyi template'te birçok kez yazarsın ya da uzun bir zinciri tekrar tekrar hesaplıyorsun. `@let` (v18.1'den beri stabil) template içinde yerel bir değişken tanımlamana izin verir; kod hem kısalmış hem okunur.

Temel Kullanım

```
HTML

@let adSoyad = kullanıcı().ad + ' ' + kullanıcı().soyad;

<h2>{{ adSoyad }}</h2>
<p>Profil: {{ adSoyad }}</p>
```

Burada `kullanıcı().ad + ...` ifadesini bir kez yazıp `adSoyad` olarak tekrar tekrar kullandık.

Hesaplama Tekrarını Önleme

```
HTML

@let toplam = sepet().reduce((a, u) => a + u.fiyat, 0);

<p>Ürün sayısı: {{ sepet().length }}</p>
<p>Toplam tutar: {{ toplam }} TL</p>
@if (toplam > 500) {
  <p>Ücretsiz kargo kazandın!</p>
}
```

Kapsam (Scope)

Bir `@let` değişkeni, tanımlandığı yerden itibaren o template bölgesinde ve iç bloklarında görünür. Örneğin bir `@for` içinde tanımladığın `@let` sadece o döngü gövdesinde geçerlidir.

Önemli: `@let` Salt-Okunurdur



Değer atayamazsın

`@let` ile tanımlanan değişkene template'te **yeniden değer atayamazsın** (event içinde bile). O, bağlandığı ifadeye göre otomatik güncellenen, salt-okunur bir kısayoldur. Durum (state) tutmak için signal kullan; `@let` sadece okumayı sadeleştirir.

Ne Zaman Kullanmalı?

- Aynı ifadeyi template'te birden çok kez kullanıyorsan.

- Uzun/iç içe bir ifadeyi okunur bir isimle kısaltmak istiyorsan.
- Bir `async` sonucu veya hesabı tek yerde tutup birçok yerde kullanmak istiyorsan.

🔗 Mini Quiz

1. `@let` ne işe yarar?
2. `@let` değişkenine template'te yeniden değer atayabilir misin?
3. Bir `@for` içinde tanımlanan `@let` nerede görünür?
4. Durum (state) tutmak için `@let` mi yoksa signal mı kullanılır?

🔗 Cevaplar

1. Template içinde okunur, yerel (salt-okunur) bir değişken tanımlar; tekrarı önler.
2. Hayır; salt-okunurdur, bağlı olduğu ifadeye göre otomatik güncellenir.
3. Sadece o döngü gövdesinde (tanımlandığı bölge ve iç blokları).
4. Durum için signal; `@let` yalnızca okumayı sadeleştirir.

⊗ Anti-Patterns

- ⊗ `@let` 'i state gibi kullanmaya çalışmak. Atanamaz; durum signal'de tutulur.
- ⊗ Her küçük ifade için `@let` açmak. Sadece tekrar eden/uzun ifadelerde değer katar; abartma.
- ⊗ Kapsam dışında kullanmak. Bir blokta tanımlayıp dışında çağırırsan bulunamaz.

✅ Bu Dersten Çıkarılacaklar

- ✓ `@let ad = ifade;` ile template içinde okunur yerel değişken tanımlarsın.
- ✓ Tekrar eden veya uzun ifadeleri sadeleştirir, okunabilirliği artırır.
- ✓ Salt-okunurdur; bağlı olduğu ifadeye göre otomatik güncellenir, elle atanamaz.
- ✓ Kapsamı tanımlandığı bölge ve iç bloklardır.
- ✓ State için signal; `@let` sadece görüntülemeyi kolaylaştırır.



Ders 4.6

input(), output(), model()

Component'ler birbirleriyle konuşur: ebeveyn (parent) çocuğa (child) **veri gönderir**, çocuk ebeveyne **olay bildirir**. Modern Angular bunu signal tabanlı üç fonksiyonla yapar: `input()`, `output()`, `model()`.

input() — Ebeveynden Çocuğa Veri

Çocuk component bir **girdi** tanımlar; ebeveyn property binding ile değer geçer. Girdi bir signal'dir; değerini `()` ile okursun:

TYPESCRIPT

```
// child: kart.ts
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-kart',
  template: `<h3>{{ baslik() }}</h3><p>{{ adet() }} adet</p>`,
})
export class Kart {
  baslik = input.required<string>(); // zorunlu girdi
  adet = input(0); // varsayılanlı (opsiyonel) girdi
}
```

HTML

```
<!-- parent template -->
<app-kart [baslik]="urun().ad" [adet]="urun().stok" />
```

`input.required<T>()` zorunlu girdidir (ebeveyn vermek zorundadır); `input(varsayılan)` opsiyoneldir. Girdiler salt-okunurdur — çocuk onları değiştirmez, sadece okur.

output() — Çocuktan Ebeveyne Olay

Çocuk bir **olay** yayar (`emit`); ebeveyn event binding ile dinler:


```
// child: kart.ts
import { Component, output } from '@angular/core';

@Component({
  selector: 'app-kart',
  template: `<button (click)="sec.emit(id())">Seç</button>`,
})
export class Kart {
  id = input.required<number>();
  sec = output<number>(); // number yayan bir olay
}
```

```
<!-- parent -->
<app-kart [id]="urun().id" (sec)="urunSec($event)" />
```

Ebeveynde `$event`, çocuğun `emit` ile gönderdiği değerdir.

model() — İki Yönlü (Two-Way)

`model()` bir girdi + çıktıyı birleştirir; ebeveyn `[]` ile iki yönlü bağlar. Çocuk değeri hem okur hem güncelleyebilir:

```
// child: sayac.ts
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-sayac',
  template: `
    <button (click)="deger.set(deger() - 1)">-</button>
    <span>{{ deger() }}</span>
    <button (click)="deger.set(deger() + 1)">+</button>
  `,
})
export class Sayac {
  deger = model(0); // iki yönlü bağlanabilir
}
```

```
<!-- parent: değer iki yönde de senkron -->
<app-sayac [(deger)]="adet" />
<p>Ebeveyndeki adet: {{ adet() }}</p>
```

Eski `<code class="inline">@Input/@Output</code>` ile İlişki

Eski (decorator)	Modern (fonksiyon)
<code>@Input() x: string</code>	<code>x = input<string>()</code>
<code>@Input({ required: true })</code>	<code>x = input.required<string>()</code>
<code>@Output() y = new EventEmitter()</code>	<code>y = output<...>()</code>
(yoktu, elle kurardın)	<code>z = model(0)</code> (two-way)

❗ Neden modern yol?

Signal tabanlı `input/output/model` daha az kod, daha iyi tip güvenliği ve reactivity ile **tam uyum** sağlar (zoneless dünyada idealdir). Eski `@Input/@Output` 'u sadece eski kodları tanımak için bil.

🔍 Mini Quiz

1. Ebeveynden çocuğa veri geçmek için ne kullanılır?
2. Zorunlu bir girdiyi nasıl tanımlarsın?
3. Çocuktan ebeveyne olay bildirmek için ne kullanılır ve nasıl tetiklenir?
4. `model()` neyi sağlar ve template'te nasıl bağlanır?

🔑 Cevaplar

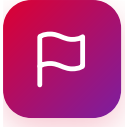
1. `input()` (ebeveyn `[girdi]="..."` ile geçer).
2. `input.required<T>()`.
3. `output<T>()` ; çocuk `this.oyay.emit(deger)` ile tetikler, ebeveyn `(olay)` ile dinler.
4. İki yönlü bağlama; ebeveyn `[(model)]= "deger"` ile bağlar.

⊗ Anti-Patterns

- × **Girdiyi parantezsiz okumak.** `baslik` değil `baslik()` ; girdiler signal'dir.
- × **Çocukta girdiyi değiştirmeye çalışmak.** `input()` salt-okunur; değişim gerekiyorsa `model()` kullan veya `output` ile ebeveyne bildir.
- × **Yeni kodda `@Input/@Output` ezberlemek.** Modern yol `input/output/model` ; eskiyi sadece tanı.

☑ Bu Dersten Çıkarılacaklar

- ✓ `input()` : ebeveyn → çocuk veri; `input.required<T>()` zorunlu, `input(varsayılan)` opsiyonel; signal olarak okunur.
- ✓ `output<T>()` : çocuk → ebeveyn olay; `emit(deger)` ile yayılır, `(olay)` ile dinlenir.
- ✓ `model()` : iki yönlü; ebeveyn `[(x)]` ile bağlar.
- ✓ Bunlar eski `@Input/@Output` 'un modern, signal tabanlı karşılığıdır.



Mini Proje: Angular ToDo App

Bu modülün her parçasını tek bir uygulamada birleştirelim: **standalone component**'ler, **signal + computed**, **dört binding çeşidi**, **@for** / **@empty** ve **input()** / **output()**. Klasik bir yapılacaklar listesi yapacağız: görev ekle, tamamla, sil, kalan sayısını göster.

Adım 0 — Hazırlık

BASH

```
ng new todo-app --style=scss --ssr=false
cd todo-app
ng g interface models/gorev
ng g c todos
ng g c gorev-item
```

Adım 1 — Model (models/gorev.ts)

TYPESCRIPT

```
export interface Gorev {
  id: number;
  baslik: string;
  tamamlandi: boolean;
}
```

Adım 2 — Çocuk Bileşen (gorev-item.ts)

Tek bir görev satırını gösterir; **input** ile görevi alır, **output** ile “değiştir/sil” olaylarını ebeveyne bildirir:

```

import { Component, input, output } from '@angular/core';
import { Gorev } from '../models/gorev';

@Component({
  selector: 'app-gorev-item',
  template: `
    <li [class.bitti]="gorev().tamamlandi">
      <label>
        <input type="checkbox"
          [checked]="gorev().tamamlandi"
          (change)="degis.emit(gorev().id)" />
        {{ gorev().baslik }}
      </label>
      <button (click)="sil.emit(gorev().id)">Sil</button>
    </li>
  `,
  styles: `
    .bitti { text-decoration: line-through; color: #999; }
    li { display: flex; gap: .5rem; align-items: center; }
  `,
})
export class GorevItem {
  gorev = input.required<Gorev>();
  degis = output<number>();
  sil = output<number>();
}

```

Adım 3 — Ebeveyn Bileşen (todos.ts)

Görev listesini signal'de tutar; `computed` ile kalanı hesaplar; çocuktan gelen olaylara göre listeyi günceller:

```

import { Component, signal, computed } from '@angular/core';
import { Gorev } from './models/gorev';
import { GorevItem } from './gorev-item/gorev-item';

@Component({
  selector: 'app-todos',
  imports: [GorevItem],
  template: `
    <h2>Yapılacaklar</h2>

    <input #kutu placeholder="Yeni görev..."
      (keyup.enter)="ekle(kutu.value); kutu.value = ''" />
    <button (click)="ekle(kutu.value); kutu.value = ''">Ekle</button>

    <ul>
      @for (g of gorevler(); track g.id) {
        <app-gorev-item
          [gorev]="g"
          (degis)="toggle($event)"
          (sil)="sil($event)" />
      } @empty {
        <li>Henüz görev yok. Yukarıdan ekle.</li>
      }
    </ul>

    <p>{{ kalan() }} görev kaldı (toplam {{ gorevler().length }})</p>
  `,
})
export class Todos {
  gorevler = signal<Gorev[]>([]);
  kalan = computed(() =>
    this.gorevler().filter((g) => !g.tamamlandi).length
  );
  private sonId = 0;

  ekle(baslik: string) {
    const t = baslik.trim();
    if (!t) return;
    this.gorevler.update((liste) => [
      ...liste,
      { id: ++this.sonId, baslik: t, tamamlandi: false },
    ]);
  }

  toggle(id: number) {
    this.gorevler.update((liste) =>
      liste.map((g) =>
        g.id === id ? { ...g, tamamlandi: !g.tamamlandi } : g
      )
    );
  }

  sil(id: number) {
    this.gorevler.update((liste) => liste.filter((g) => g.id !== id));
  }
}

```

```
}  
}
```

Adım 4 — Kök Bileşene Ekle (app.ts & app.html)

TYPESCRIPT

```
import { Component } from '@angular/core';  
import { Todos } from './todos/todos';  
  
@Component({  
  selector: 'app-root',  
  imports: [Todos],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css'],  
})  
export class App {}
```

HTML

```
<!-- app.html -->  
<h1>ToDo App</h1>  
<app-todos />
```

Adım 5 — Çalıştır

BASH

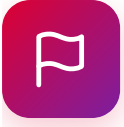
```
ng serve  
# localhost:4200 – görev ekle, kutucukla tamamla, sil; "kaldı" sayısı anında güncellensin
```

🔗 Bu projede ne kullandın?

Standalone bileşenler + **imports**, **signal** (liste) + **computed** (kalan), dört binding çeşidi (**{{ }}**, **[class.bitti]**, **(click)/(change)/(keyup.enter)**, **@for** + **track** + **@empty**, ve **input.required** / **output** ile ebeveyn-çocuk iletişimi. Modülün tamamı tek uygulamada!

☑ Bu Dersten Çıkarılacaklar

- ✓ İki standalone bileşen (ebeveyn `Todos` + çocuk `GorevItem`) kurdun.
- ✓ Liste `signal` 'de, kalan sayısı `computed` ile; `update` / `filter` / `map` ile değişmezlik (immutability).
- ✓ `@for ... track g.id` + `@empty` ile listeyi ve boş durumu yönettin.
- ✓ `input.required` ile veri indirdin, `output` + `emit` ile olay yukarı taşıdın.
- ✓ Modül 4'ün tüm kavramları gerçek, çalışan bir uygulamada birleşti.



Artık Angular'ın kalbini biliyorsun: component yazıyor, aralarında veri taşıyor ve modern template sözdizimini kullanıyorsun.

Öğrendiklerin

- **Component anatomisi:** `@Component` (selector/template/style), scoped stiller, lifecycle (`ngOnInit`).
- **Standalone:** NgModule yok; bağımlılıklar kendi `imports` 'unda.
- **4 binding:** `{{ }}`, `[prop]`, `(olay)`, `[(model)]`.
- **Control flow:** `@if`, `@for` (`track` + `@empty`), `@switch`.
- **@let**: template içi salt-okunur yerel değişken.
- **İletişim:** `input()` / `output()` / `model()`.

❓ Sırada ne var?

Modül 5'te bu modülde her yerde kullandığımız **Signals & Reactivity**'yi derinlemesine ele alacağız: `signal`, `computed`, `effect`, `linkedSignal`, `resource` ve signal tabanlı sorgular. Angular'ın zoneless dünyasının motoru tam olarak burası.

💡 Kendini test et

Şunları cevaplayabiliyor musun? “Standalone ne demek? Dört binding çeşidi nedir? `@for` 'da `track` neden zorunlu? `input` ile `model` farkı ne?” Cevaplayabiliyorsan Modül 5'e hazırsın.

Takıldığın her yeri sor — hazır olduğunda Modül 5'e geçeriz. Kolay gelsin!